

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Matias Prieto Escobar

11 de septiembre de 2026

23:35

1. Introducción

El análisis de algoritmos suele apoyarse en la notación asintótica para comparar su eficiencia teórica, pero esta no siempre predice su comportamiento real sobre hardware concreto. En este trabajo se estudian experimentalmente dos problemas clásicos: el ordenamiento de arreglos unidimensionales de números enteros, mediante los algoritmos MERGESORT, QUICKSORT, PATIENCESORT y `std::sort` y la multiplicación de matrices cuadradas, mediante el algoritmo NAIVE ($O(n^3)$) y el algoritmo de STRASSEN ($O(n^{\log_2 7})$). El objetivo es medir tiempo de ejecución y uso de memoria de cada algoritmo bajo distintos tamaños de entrada, tipos de dato (ordenado, invertido, aleatorio; denso, diagonal, disperso) y dominios de valores, para luego contrastar estos resultados con su complejidad teórica y entender en qué medida esta última describe o no su desempeño práctico.

2. Implementaciones

El código fuente completo de esta tarea se encuentra en:

<https://github.com/Rextrere800/tarea1-INF221>

3. Entorno experimental

Los experimentos se ejecutaron en un equipo con procesador AMD Ryzen 7 5700X, 24 GB de RAM a 2133 MT/s, bajo Windows 11 (versión 25H2, compilación 26200.9445). El código se compiló con g++

16.2.0 (MSYS2, Rev3) usando el estándar C++17 y el flag de optimización -O2 (g++ -O2 -std=c++17), consistente con el `makefile` incluido en el repositorio.

Los datasets utilizados corresponden a los descritos en el enunciado de la tarea: para ordenamiento, arreglos de $n \in \{10, 10^3, 10^5, 10^7\}$ elementos, con tipo $\in \{\text{ascendente, descendente, aleatorio}\}$ y dominio $\in \{D1, D7\}$; para multiplicación de matrices, matrices cuadradas de $n \in \{16, 64, 256\}$ ¹, con tipo $\in \{\text{densa, diagonal, dispersa}\}$ y dominio $\in \{D0, D10\}$. En ambos casos se generaron 3 muestras aleatorias independientes por combinación de parámetros.

4. Resultados y Análisis

4.1. Multiplicación de matrices

La Figura 1 muestra el tiempo de multiplicación para matrices densas, dominio D0. Se observó el mismo patrón en las demás combinaciones de tipo (densa, diagonal, dispersa) y dominio (D0, D10), lo que es esperable: ni NAIVE ni la implementación de STRASSEN utilizada explotan la estructura de la matriz (ambas almacenan y operan sobre `vector<vector<int>>` denso independientemente de los ceros presentes).

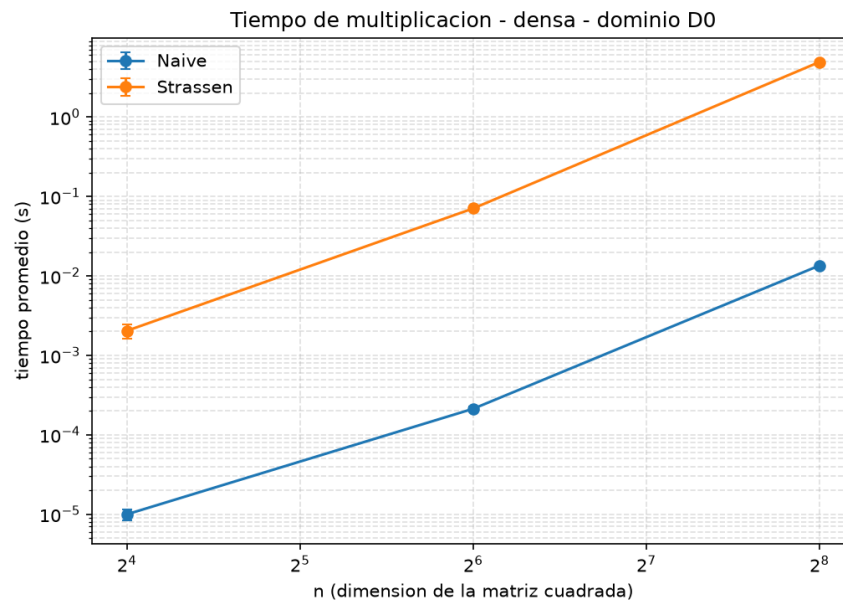


Figura 1: Tiempo de multiplicación, matrices densas, dominio D0.

Sorprendentemente, STRASSEN resultó entre 200 y 364 veces más lento que NAIVE en todo el rango medido ($n = 16$ a $n = 256$), pese a tener menor complejidad asintótica ($O(n^{\log_2 7}) \approx O(n^{2,807})$ frente a $O(n^3)$) (los cálculos fueron realizados extrayendo los valores de los archivos csv). Más aún, al comparar la tasa de crecimiento entre $n = 64$ y $n = 256$ (un aumento de $4 \times$ en n), NAIVE creció $63,6 \times$ –prácticamente idéntico a los $64 \times$ que predice $O(n^3)$ –, mientras que STRASSEN creció $69,7 \times$, **superando** el $54,7 \times$ que

¹Se excluyó $n = 1024$ (2^{10}) de las mediciones, con autorización del curso.

predice su propia complejidad teórica $O(n^{2,807})$. Esto indica que el comportamiento de STRASSEN en este rango no está dominado por la cantidad de operaciones aritméticas (que sí sigue de cerca su cota asintótica), sino por las instancias creadas en recursion, generando multiples `vector<vector<int>`.

El uso de memoria (Figura 2) confirma este diagnóstico: STRASSEN usa consistentemente entre 6.9x y 9.1x más memoria que NAIVE para el mismo n , coherente con la gran cantidad de matrices temporales que crea en cada nivel de recursión. Además, la memoria fue idéntica (desviación estándar 0.0000 MB) entre los tipos densa/diagonal/dispersa para cada combinación (algoritmo, n), lo cual es esperado dado que ninguna de las dos implementaciones ajusta su estructura de datos según la dispersión de la matriz.

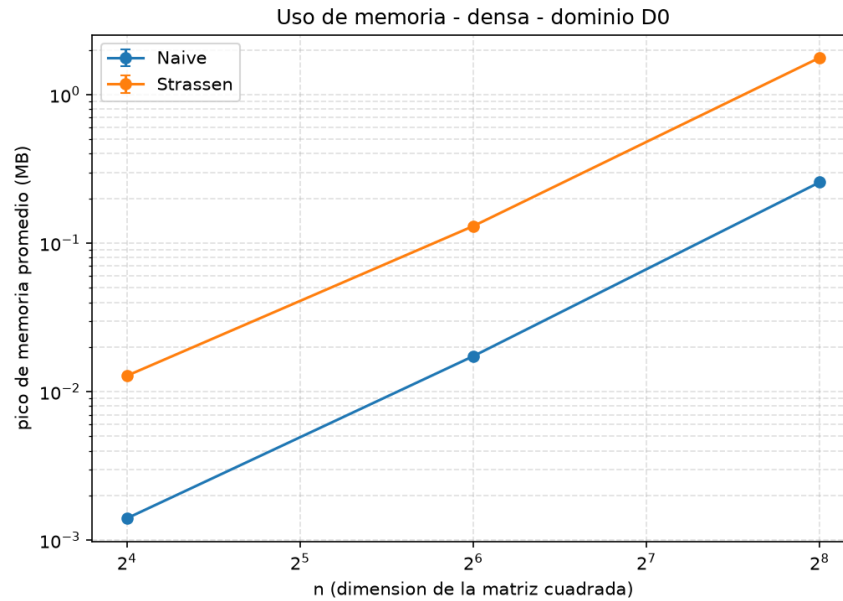


Figura 2: Uso de memoria, matrices densas, dominio D0.

Este resultado ilustra el objetivo central del experimento: la notación asintótica describe el comportamiento cuando $n \rightarrow \infty$, pero no garantiza mejor desempeño para los tamaños de entrada efectivamente utilizados en la práctica, especialmente cuando la implementación introduce overheads (aquí, de gestión de memoria) que la notación no captura.

4.2. Ordenamiento de arreglos

Las Figuras 3 y 4 muestran el tiempo de ordenamiento para $n = 10^7$ en los casos ascendente y aleatorio, comparando los dominios D1 (10 valores posibles, con altísima tasa de repetición) y D7 ($10^7 + 1$ valores posibles, casi todos distintos).

QUICKSORT es, con diferencia, el caso más interesante. En D1, su partición de 3 vías absorbe grandes bloques de elementos iguales al pivote en cada llamada, lo que lo acerca a un comportamiento casi lineal: a $n = 10^7$ termina en apenas 0,04–0,10 s en los tres tipos de entrada, superando incluso a `std::sort`. En D7, en cambio, el comportamiento depende fuertemente del orden de los datos: en el caso *aleatorio* toma 0,75 s (solo 7,4× más que D1, la diferencia esperable por la pérdida del beneficio de los duplicados), pero

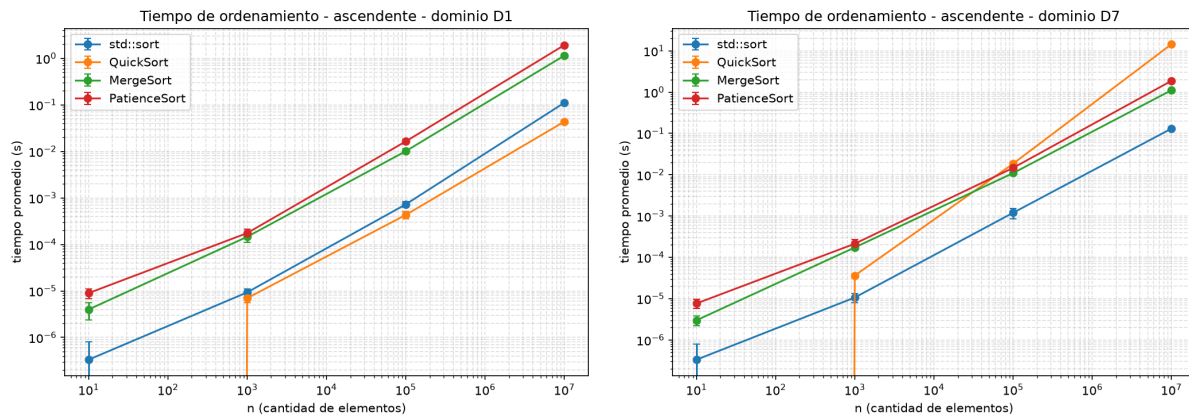


Figura 3: Tiempo de ordenamiento, caso ascendente, dominios D1 y D7.

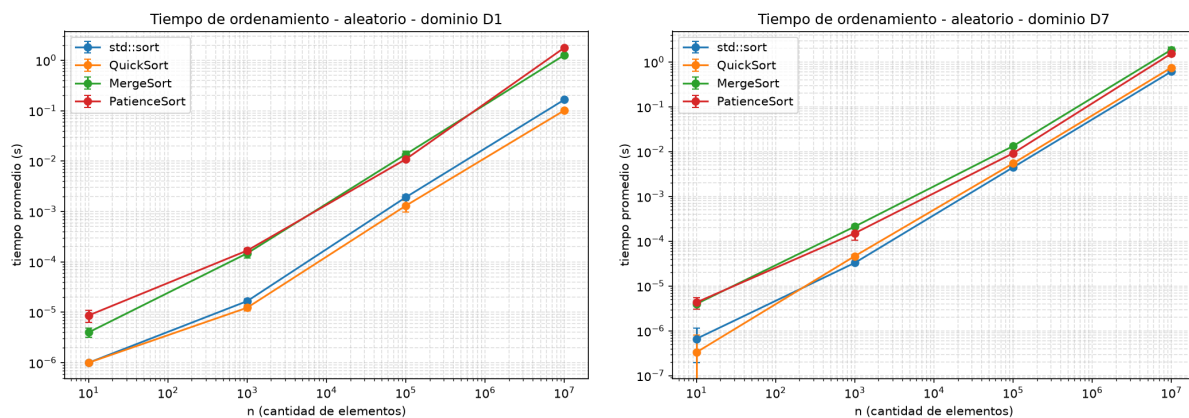


Figura 4: Tiempo de ordenamiento, caso aleatorio, dominios D1 y D7.

en los casos *ascendente* y *descendente* explota a 14,4 s y 10,3 s respectivamente —331× y 265× más lento que su contraparte D1—, muy por encima del resto de los algoritmos.

Esto no es una falla del pivote fijo del QUICKSORT clásico (que la mediana de tres ya evita), sino una variante del fenómeno conocido en la literatura como "*median-of-three killer*" [1]: la técnica de mediana de tres, al colocar el pivote correcto en la última posición mediante intercambios directos, puede desplazar el valor máximo del rango justo al centro de un arreglo que sigue ordenado, generando particiones muy desbalanceadas en niveles sucesivos de la recursión. El efecto solo se manifiesta cuando los valores son casi todos distintos (D7); con alta repetición (D1) la partición de 3 vías elimina la estructura ordenada antes de que el desbalance se acumule, lo que explica por qué D1 nunca sufre este problema sin importar el tipo de entrada.

MERGESORT y PATIENCESORT, en cambio, muestran tiempos consistentes entre tipos y dominios (variación menor a 2× en todos los casos), como corresponde a algoritmos cuya complejidad no depende del orden ni de la repetición de los datos de entrada.

Este resultado refuerza el mensaje central del experimento: una optimización que en teoría debería blindar contra el peor caso clásico (mediana de tres contra el pivote fijo) puede introducir, en la práctica,

Cuadro 1: Tiempo promedio de QuickSort a $n = 10^7$ (segundos).

Tipo	D1	D7
Ascendente	0.044	14.440
Descendente	0.039	10.296
Aleatorio	0.101	0.754

una vulnerabilidad distinta y menos conocida, que solo se manifiesta bajo condiciones específicas de los datos de entrada.

5. Conclusiones

Los resultados confirman que la complejidad asintótica no basta para predecir el desempeño práctico. STRASSEN, pese a su menor complejidad teórica ($O(n^{2,807})$) frente a $O(n^3)$ de NAIVE), resultó hasta $364\times$ más lento en el rango medido, debido al overhead de asignación de memoria de su recursión. De forma análoga, la optimización de mediana de tres en QUICKSORT, diseñada para evitar el peor caso clásico del pivote fijo, introdujo una vulnerabilidad distinta (el fenómeno conocido como "*median-of-three killer*" [1]): en el dominio D7, tardó $331\times$ y $265\times$ más en los casos ascendente y descendente respectivamente que en D1, mientras que en el caso aleatorio la diferencia fue de solo $7,4\times$. En ambos experimentos se repite el mismo patrón: una optimización teóricamente sólida puede fallar en la práctica bajo condiciones de entrada específicas que la notación asintótica no captura, reforzando la necesidad de validación experimental más allá del análisis teórico.

Referencias

- [1] M. Douglas McIlroy. «A Killer Adversary for Quicksort». En: *Software: Practice and Experience* 29.4 (1999), págs. 341-344.